

Charles Senftleben

Senior Engineering Manager at American Express

System Design, Sep 25, 2026

Senior Software Engineer

Remote - United States

Personal Info

Phone number

4804064935 (other)

+14804064935 (other)

Email

morimar@gmail.com

Address

Phoenix, AZ (home)

Social Media

--

Websites

<https://www.linkedin.com/in/charles-senftleben> (other)

Source

Referred by Matt Syp

Responsibility

Recruiter

Andrew Baker

Coordinator

Ashley Cadely

Candidate Application

Job Post Questions

Have you ever worked for Datavant or one of our affiliates? (Ciox Health, ArroHealth, IOD, ECS, Healthport, Medal, MedSave, Chart One, Discovery Health, Documentation Solutions, Smart Corp, Care Communications, Aetion, Digital Owl, Ontellus)?

No

Are you at least 18 years of age?

Yes

Do you have a High School Diploma or Equivalent (GED)?

Yes

Are you authorized to work lawfully in the United States?

Yes

Will you now or in the future require Datavant to sponsor your employment authorization (for example, through an H-1B or similar visa)?

No

Will you be able to work in the location listed in the job description upon day one of your employment?

Yes, I can work in the location listed or will self-fund relocation to do so.

How did you hear about this job?

Employee Referral

If you selected "Employee Referral", please list the referring employee's full name.

matt.syp@datavant.com

LinkedIn Profile

<https://www.linkedin.com/in/charles-senftleben>

I consent to receive text messages regarding my job application status, interview updates, and future job opportunities from Datavant. You can opt out any time by texting STOP. For assistance, text HELP. Message & data rates may apply; Messaging frequency may vary. Visit www.datavant.com/privacy-policy to see our privacy policy.

Yes, I opt in to receive text messages.

By submitting your application you confirm that the information you provide is accurate and complete to the best of your knowledge, and that you will promptly inform us if you become aware of any errors or inaccuracies.

I confirm

Will you now or in the future require Datavant to participate in a government program to maintain your work authorization (such as completing training verification for OPT or STEM OPT)?

No

Do you currently work for Datavant, CVS or any affiliates of either company?

No

Interview Prep

System Design Interview Guide

Goals of the Interview

In this interview, we're primarily trying to learn three things about the candidate:

1. Will they explore a problem to make sure they're solving the right thing
2. Given the problem they've fleshed out, will they pick technologies that solve it well
3. For those technology choices, can they explain why they work well and what tradeoffs they're making against other technologies

Note that for all of these we're not expecting perfect knowledge of the entire problem and technology landscape. Instead we want to see that they can identify the important considerations in making a choice and have a way to evaluate them. Saying "I don't know exactly what database I'd use here, but I'd want it to have these querying capabilities" can be a better answer than picking something specific without considering if it's appropriate!

Interview Etiquette

Mostly, use common sense. Introduce yourself and your pronouns, and make an effort to understand the candidate's name. Check with them about if they need a break or water if Greenhouse says this isn't their first interview.

One point of etiquette that does deserve special attention is that we offer to take notes for the candidate. You can use the notes to guide interviewees through the section, e.g., say "Next let's talk about database choices. What type of database would you choose" and type out "database design" in the doc to move the candidate forward.

What you need going into the interview:

- Coderpad link for candidate notes
 - Coderpad's drawing tool now uses Excalidraw so it should be a good enough tool for our purposes.
 - However, if the candidate has their own drawing tool and wants to share their screen, we've seen that work well.
- Interview kit

How to Evaluate Candidates

All passing candidates should describe a system that covers the baselines in each section. Experienced candidates should do that on their own, but it's OK if more junior candidates needed some help reaching the baselines. When scoring a candidate, if they needed your help to get a baseline or bonus point, count that as a half point instead of a full point.

The more experience a candidate has, the fewer yellow flags and more bonus points they should have. A candidate with 10 years of experience should likely have no or one yellow flags and over 5 bonus points to pass. A new grad could have 5 yellow flags and no bonus points and still pass.

No candidate should pass with a red flag.

Beyond these system design specific criteria, we're looking for Smart, Nice, and GTD in all interviews. Be sure to note that down, too, and consider it in your yes/no decision.

P2 Points (5) P3 Points (10) P4 Points (11) P5 Points (3) Yellow Flags Red Flags

No	0-3			5+	1+
P2	4-5			0-4	0
P3	5	7-10		0-3	0
P4	5	8-10	7-11	0-2	0
P5	5	9-10	9-11	2-3	0-1
					0

Understanding the question

Candidate Prompt:

We want to build a system to answer this question: of COVID diagnostic tests with a positive result, what percentage were for people that were unvaccinated, partially vaccinated, and fully vaccinated?

Known (but Held Back) Requirements for the System

The above prompt is all we give the candidate to start with. We purposefully present a vague, high-level prompt for them to explore. One of the things we're assessing is if they gather knowledge about what needs to be built before jumping in to design it.

Beyond what's in the prompt, these are additional requirements for the system they need to build. We share that with the candidate as they ask about it. It's your job as an interviewer to encourage them to do that exploration and make them comfortable enough that they can do so. The Hints section below has good ways to do that.

Ingestion

- CSVs of vaccinations administered and COVID test results will be uploaded by clinics, pharmacies, and hospitals throughout the US
 - Our best guess is that 10s of thousands of facilities will be uploading
- Someone at the facility will upload their data roughly once a day
 - That it's a regular employee doing the upload from systems already in the facility strongly pushes towards a browser-based upload
 - The file sizes will range from 1KB to 500 KB uncompressed and that it'll compress 10:1 such that it'd be 50 KB in transmission and storage
 - If we say 50 bytes per row of timestamp, vaccine identifier, patient token, and facility identifier, 500 KB is a facility uploading 100k patients in a day
 - The CSV is quite repetitive, so it should compress well.
- We can specify what's in the CSV, but the basic building blocks are patient identifier, time of procedure, COVID test result, and vaccine type administered
 - The facility can't include vaccination status in the CSV as they may not have all the doses given to determine that!
- We target [99.5% availability](#) to keep from inconveniencing facility workers too much
- There will only be individual day uploads i.e. no bulk uploads
 - If there's extra time, you can ask about handling bulk uploads as an add-on

Querying

- A vetted set of researchers can make queries against the collected data
- They're expecting to use a REST API that returns JSON to do so
- Data should be returned from the query API within 24 hours
- There are 1,000s of researchers, but fewer than 10,000
 - They'll query multiple times a day but not 1000s of times
- They need the API to be able to give vaccine status percentages broken down by day and state in which the test was administered for arbitrary date ranges
 - We add state to make it have more than just time as a filtering criteria, but don't add more than that to keep it simple for the conversation
- Vaccination status is determined by a library that takes in the date you want to know status on and the set of vaccines a patient has received and the dates they were administered
- We target [99% availability](#) since researchers can deal with a little downtime
- We want queries to return in 30 seconds or fewer at the 99th percentile

Hints for the Candidates

Common Questions from Candidates

Routinely good candidates ask questions along these lines. In order to standardize responses try to absorb the answers listed here. Some of these questions we can straightforwardly answer. Other questions we should turn back on the candidate to gain more signal.

Straightforward Questions

These questions we can answer without too much digging. You can respond straightforwardly if a candidate asks you something along these lines.

What population are we covering?

US pop, ~350 million people (don't tell them but something for you to keep in mind: not necessarily 350 million data points since people can test a lot)

How frequently is the data delivered?

We can expect that data uploaders will upload no more than 1/day, but could whatever frequency they choose (and they can upload historical data up to the start of the project, which could be for example many months - the historical data part I usually don't spell out super explicitly and throw it in as a curveball later if they expect all data to come in in chronological order)

Also, data should be available in the system within 24 hours (we could probably decrease this amount but basically the point is we don't need to have a real time ingestion system)

How is the data joined/do we get PII/how do we identify a person?

Ask if the candidate has done the debug interview and if they remember the tokens from the problem. Say that the Datavant tokenization process has run as part of exporting the CSVs, so it contains anonymized Datavant IDs that are perfectly uniquely identifying.

More Complex Questions

These questions are opportunities to get more signal on the candidate. Share the information with the candidate, but check their intuitions first or pause to question their understanding once you have delivered the hint.

How is the data formatted?

First Response: "What are your expectations for how the data would be formatted?"

Does the candidate understand that medical institutions aren't the most tech savvy so they realistically aren't going to set up a real time system or be able to plug into our api?

Ultimately: We can say they have CSVs that we specify a format for (aka we tell them the headers for the CSV)

What data fields come in?

First response: "What would you need?"

Answer: If they expect that vax status is coming in (usually during the db schema part) then I'll say we get vaccination events and test events but not vax status. The vaccine provider doesn't know about vaccines administered elsewhere, so they can't provide status.

Extra: Suppose there is a "bad actor" of someone who is getting extra vaccines and they might be dishonest with their vax provider? This is mostly to demonstrate that a vax provider might not know the true vax status of an individual, instead they only have data about what vaccines they have administered

How do we know the definition of fully/partially vaccinated?

Respond that the definition can change over time (e.g. after 9 months of the second vax you may need a booster), but a rules engine is available for us to use so we don't need to create the algo, we just need the data available to pass to that library.

Check: Does the candidate recognize there is an undefined amount of vaccinations possible, so they can't just have two columns for vaccine 1 and vaccine 2 on a patient db row?

Example Script for Interview

You don't need to follow this interview script perfectly, but to give candidates a fair shot we try to structure the interview with these questions. Try to guide the interview along these pathways.

Introduction (5 mins)

- Greeting: "Hello! I'm Mariel, a software engineer at Datavant. I use she/her pronouns. I'm looking forward to our interview today."
- Comfort Check: "Before we begin, do you need any water or anything else to feel comfortable?"
- Brief Introduction: "I'm part of the payer product pod, focusing on building features for insurers, particularly for chart retrieval requests."
- Ask the candidate to introduce themselves
- Summary of goals: "What I'm really curious about understanding today is how you build software in the real world, so what are the factors you take into consideration and how do you ultimately arrive at a decision."
- Problem Introduction: "We want to build a system to answer this question: of COVID diagnostic tests with a positive result, what percentage were for people that were unvaccinated, partially vaccinated, and fully vaccinated?"

Design Discussion (45-50 mins)

Always start with Requirements Gathering. Doing that will set up the candidate for success in the other sections. If they're missing important information but want to move on, prompt them a bit to explore more. Once they've gathered enough or haven't heeded the warning, the candidate can move through the other sections as they see fit.

For each of the sections, we have a baseline of what we expect, bonus points for good things beyond that, yellow flags for things that aren't disqualifying unless we have many, and red flags for things that are disqualifying on their own.

There's much less scripted direction for the later sections. Since this interview works best as an exploration and discussion, it's hard to script it out. Instead, use the baselines to make sure they cover everything and let the candidate lead otherwise.

Requirements Gathering (At least 10 mins)

- Say: "Imagine I'm a PM with a technical bent. Ask me everything you'd want to know to be able to build this system. The more you find out here, the better your design will match what we need"
- Make clear: "We don't need all the details for each component of the system, but we should know all the major components and how they work together to solve the problem by the end of the discussion." If they're taking a long time with requirements gathering, you can say something like "let's get started with the components and I can always answer any more questions that come up."
- "What are the key components of a researcher-facing COVID-19 database at a high level?"
- This is where the candidate should be discovering [the requirements for the system](#).

Out of this section, the candidate should identify three high-level components: something to upload the data, something to serve the data up to the researchers, and something between the two to store it. From there, they can steer the conversation to fill in details on whatever's most interesting to them for the remaining time to the questions at the end.

Baseline

- Gets the major system requirements: accepting uploads from many facilities, queries from fewer researchers, and scalability/availability/latency

Bonus Points

- Discovers additional nuanced requirements like vaccination status not being uploaded and exact query interface

Yellow Flags

- Immediately diving into designing the system without gathering requirements or continuing to do so after being prompted to gather more requirements

Data Upload

For the upload, a web portal is easiest to make work well. Given the requirement that the data come from 10s of thousands of individual facilities with unknown capabilities on the uploading machine, the one common denominator is a browser. A desktop app will be hard to get installed and usable, and email doesn't provide enough feedback on if it's working or not. If they choose SFTP or an S3 GUI, they'll need to pair that with an explanation of how they'll install it, instruct users, and give feedback. If they haven't figured out the requirement that the data is being uploaded by individuals at the facility, steer them there as they figure out this component.

Baseline

- Web portal / alternate user-friendly upload interface
- Upload is stored in an object-storage system like S3
- DB insertion is performed as a background process, not in the same process as the upload

Bonus Points

- Mentioning auth for users, how to provide support for creating and managing them
 - Extra points for thinking about using an auth provider like Auth0

- Having the upload be directly to a signed URL in S3 or an S3-like object store so we don't store the bytes at all for it and can still allow upload if the database is down
- Detecting duplicates by hashing the whole upload and/or unique constraints in the db
- Thinking about good UX like immediate, helpful error messages, clear confirmation on completion, or asynchronous notifications of processing failures
- Considering storing the raw uploads long term to allow reprocessing if needs change
- Mentioning a load balancer in front of the portal with a couple nodes behind it for zero-downtime deploys
 - If they do start talking about specific load balancers or k8s setups to do this, say you're glad they're thinking about that but that we don't need to dig into specifics
- Asking to minimize data uploaded to make privacy breaches less bad, asking for tokenized PII, mentioning encrypting the uploaded files at rest

Yellow Flags

- Inserting directly into the db as part of the upload
- Storing the upload on the local filesystem for background processing
- Dropping the whole upload into a queue for background processing
 - Even worse if they don't consider the size of the uploads or the message size limit on the queue
- Not considering the requirement around latency between upload and availability
 - The requirement is 24 hours, so it's not hard to meet. But assuming that while doing the insertions is a yellow flag
- Does not justify their technical choices

Red Flags

- Not being able to articulate why they're choosing a particular path when prompted by the interviewer
- Ignoring interviewer guidance

Database Design

As with the upload, the structure of this problem funnels all answers to a specific solution. In the case of storage, that's a database. The storage must:

- Handle the insertion of data from lots of files from small to large
- Allow the lookup of test results filtered by date and state
- Support the calling of a library to determine vaccination status for those test results
- Store rows in the low billions

That can be done with traditional relational databases, distributed databases, and likely other kinds of databases. It's hard to do by querying the uploaded files directly or anything more esoteric. If a candidate starts going down that route, steer them to a database with the above constraints. If they have compelling answers to all of them, then explore the new storage with them. Maybe they have a better way!

When talking about the database, get them to describe the schema, insertion, and queries if they don't do that on their own.

Baseline

- Description of the schema
- How the upload will insert into the db
- How to calculate the percentages from the prompt using the schema
- [Rough performance characteristics of insertion and queries](#)

Bonus Points

- Proactively mentioning indexes that will be used for lookup
- Calculating data size and format to think about schema usage, query times
- Realizing that vaccination status must be calculated from vaccination data and structuring the data appropriately
 - Precalculation of vaccine status or days can be appropriate here, and if they do that in a way that ensures the precalculation stays up to date as more data is ingested, that's a big bonus point
- Rate limiting insertions in the background such that a rush of large uploads don't completely tank query performance
- Logic on how to detect/handle duplicate rows
- Specifies a clean pre-calculation or caching design

Yellow Flags

- Not explaining what constraints are driving them to a particular database choice
- Unable to explain the query they'd run

Red Flags

- Not being able to articulate why they're choosing a particular path when prompted by the interviewer
- Ignoring interviewer guidance

Query API

The final component is an API for the researchers to query the database. We've specified it as a REST API in the requirements, so if they try to create a UI, steer them back to REST.

Baseline

- High-level description of an endpoint that takes in date ranges and an optional state and returns percentage results by day
- Pseudo-code walking through what queries that endpoint would run against the db and how it would transform them into the results

Bonus Points

- Thinking through how we'd provide auth to the researchers
- Considering how the researchers would use the API, what we could do to make it more ergonomic for them or seeing if want UX like UI charts
- Giving a rough estimation of the time for queries, thinking about indexes and data size

Conclusion (5-10 mins)

- If the candidate gets through all the components and we have a path through the whole system, summarize how it would work e.g. "we upload to S3, get notified on SQS, insert from a worker, query when the researcher calls". Then say that's what you were looking for and try to get them in a good mental place for their next interview regardless of how this one went.
- "What would you like to know from me?"
- Example farewell: "We appreciate your thoughts and approach to these challenges. Good luck!"

Scorecard

Key Take-Aways

This was a mostly solid system design interview with room for improvement. Overall I'm close to a strong yes, but I don't think I can justify it with the gaps around ingestion, even after I pushed a bit.

Charles communicated clearly and quickly had a handle on important aspects of how the system needed to work. He asked good questions and clearly explained how to answer the prompt from the data structures he proposed. He suggested using the Datavant token he'd learned about in the previous round of interviewing. He brought up PII security and specifics of how to have the partner network login. The main point where I can't give him full credit is the ingestion. He proposed a web portal where a user would paste rows that closely matched his DB structures into a grid. He missed a lot of points around uploading to S3 and so on, because the primary input mechanism was essentially just direct inserts in the DB. I asked about how he'd implement his validation approach in that setup, I pushed on the size of those copy/pastes, and I brought up the concern about a new partner being added late with a lot of historical data to upload. I even misunderstood the way he planned to have the upload work and assumed out loud that he'd have files uploaded, but the best I could get out of him was that he'd allow file uploads for the historical data case and try to get people to use the direct entry instead. If he'd understood the correction and discussed a bit about how to show validation results to the users or using S3 pre-signed URLs or something, I'd probably give him a strong yes. Overall, I think Charles will be a solid addition to Datavant and I'd be happy to work with him.

P2 points: 4.5 out of 5

P3 points: 4.5 out of 10

P4 points: 8 out of 19

P5 points: 2.5 out of 15

Yellow flags: 1.5 out of 9

Red flags: 0 out of 2

Coder pad: <https://app.coderpad.io/CFNEDK27/playback#2132>

Requirements Gathering

[1] **P2** - Gets the major system requirements: accepting uploads from many facilities, queries from fewer researchers, and scalability/availability/latency

[1] **P3** - May discover additional nuanced requirements like vaccination status not being uploaded and exact query interface

[1] **P4** - Discovers non-functional requirements, such as scalability, latency, availability, privacy, and operational

constraints

- [] **P4** - Discovers team constraints that would affect velocity and/or delivery estimates
- [] **P4** - Identifies areas of technical risk or uncertainty
- [] **P5** - Considers cross-functional concerns, such as support model, operations, revenue, etc.
- [] **P5** - Considers capabilities and capacity of the team who will work on the project
- [] **P5** - Asks about the oncall/support model and/or who will own the product longer term

Data Upload

- [1] **P2** - Proposes a web portal or some alternate user-friendly upload interface
- [1] **P3** - Thinking about good UX like immediate, helpful error messages, clear confirmation on completion, or asynchronous notifications of processing failures
- [] **P3** - Upload is stored in an object storage system like S3
- [] **P3** - DB insertion is performed as a background process, not in the same process as the upload
- [1] **P4** - Mentions auth for users, how to provide support for creating and managing them
- [1] **P4** - Goes into detail about authorization or about using a specific auth provider like Auth0
- [] **P4** - Having the upload be directly to a signed URL in S3 or an S3-like object store so we don't store the bytes at all for it and can still allow upload if the database is down
- [1] **P4** - Detecting duplicates by hashing the whole upload and/or unique constraints in the db
- [] **P4** - Considering storing the raw uploads long term to allow reprocessing if needs change
- [] **P4** - Mentioning a load balancer in front of the portal with a couple nodes behind it for zero-downtime deploys
- [1] **P4** - Asking to minimize data uploaded to make privacy breaches less bad, asking for tokenized PII, mentioning encrypting the uploaded files at rest
- [] **P5** - Considers existing platform capabilities, e.g. does Datavant have data ingestion pipelines, and could we use those instead?
- [1] **P5** - Considers late, out-of-order data (upstream and downstream, does that affect UX or data products)
- [] **P5** - Addresses schema evolution, backwards compatibility (mentions of AVRO, Parquet, ORC, JSON, CSV and tradeoffs)
- [] **P5** - Reasons through the tradeoffs in a centralized model versus a distributed model
- [] **P5** - Includes operational cost or team friction in the criteria for choosing a solution

Database design

- [1] **P2** - Provides a useful description of the schema
- [0.5] **P2** - Designs how the upload will insert into the DB
- [1] **P2** - Calculates the percentages from the prompt using the schema

- [1] **P3** - Calculates the percentages from the prompt using the schema
- [] **P3** - Proactively mentioning indexes that will be used for lookup
- [] **P3** - Talks about rough performance characteristics of insertion and queries
- [] **P4** - Calculates data size and format to think about schema usage, query times
- [1] **P4** - Recognizes that vaccination status must be calculated from vaccination data and structuring the data appropriately
- [] **P4** - Rate limits insertions in the background such that a rush of large uploads don't completely tank query performance
- [1] **P4** - Provides logic for how to detect/handle duplicate rows
- [1] **P4** - Specifies a clean pre-calculation or caching design
- [1] **P5** - Precalculates vaccine status or days in a way that ensures the precalculation stays up to date as more data is ingested
- [] **P5** - Considers other use cases for the dataset and application
- [0.5] **P5** - Designs for future modifications/extensibility
- [] **P5** - Talks through how and when particular optimizations should be introduced (precomputation, caching)

Query API

- [1] **P2** - High-level description of an endpoint that takes in date ranges and an optional state and returns percentage results by day
- [1] **P3** - Pseudo-code walking through what queries that endpoint would run against the db and how it would transform them into the results
- [] **P3** - Thinking through how we'd provide auth to the researchers
- [0.5] **P3** - Considering how the researchers would use the API, what we could do to make it more ergonomic for them or seeing if want UX like UI charts
- [] **P4** - Gives a rough estimation of the time for queries, thinking about indexes and data size
- [] **P4** - Offers a logical argument for why the 30s SLA is or is not achievable
- [] **P4** - Designs a complete REST API from the requirements discovered that adheres to the principles of REST
- [] **P4** - Solves for authentication and/or discusses client ergonomics (ease of use), such as applying HATEOAS
- [] **P5** - Talks about the API contract and documentation
- [] **P5** - Considers broader aspects of API: versioning, backwards compatibility, rate limiting/abuse, extensibility
- [] **P5** - Discusses the tradeoffs and balance between ergonomics (ease of use) and reliability/risk to SLOs/SLAs

Yellow Flags: Yellow flags do not disqualify a candidate but they are warning signals. Yellow flags may be

balanced out by strong performance in other parts of the question.

Requirements Gathering

☐ Immediately dives into designing the system without gathering requirements or continuing to do so after being prompted to gather more requirements

Data upload

☐ [1] Inserts directly into the db as part of the upload

☐ [] Stores uploaded data on local filesystem

☐ [] Puts entire file contents into a queue for background processing

☐ [] Does not consider requirement around latency between upload and availability (if they assume 24 hours correctly without confirming with interviewer, that is also a yellow flag)

☐ [0.5] Does not justify their technical choices

Database design

☐ [] Does not justify DB choice or chooses a DB that does not match the prompt criteria

☐ [] Unable to explain the query they'd run

☐ [] Premature optimization UNLESS it provides clear strategic/organizational leverage

Red Flags: These are disqualifying behaviors. They indicate that the candidate would not be easy to work with, either from lack of communication or unwillingness to take opinions from other parties.

☐ [] Not being able to articulate why they're choosing a particular path when prompted by the interviewer

☐ [] Ignoring interviewer guidance

Overall Recommendation

Did the candidate pass the interview?

1. 2. 3. 4.

Charles Senftleben

M.B.A., Systems Thinker & Builder, Creative problem solver.

Phoenix, AZ

(480) 406-4935

morimar@gmail.com

<https://www.linkedin.com/in/charles-senftleben/>

EXPERIENCE

American Express, Phoenix, AZ — Senior Engineering Manager

April 2024 - Present

- Managing team within the Account Servicing group, responsible for account management within MYCA (Manage Your Customer Account), both within the United States and Internationally. Each team having 3 – 4 frontend engineers, 2 backend engineers, and 2 quality engineers.
- Directed the seamless transition from a legacy backend system to a modern internal service by orchestrating cross-functional collaboration, developing and executing a strategic release plan, and maintaining team focus and momentum, culminating in an on-time, high-impact delivery.

OpenDoor, Tempe, AZ — Engineering Manager

October 2022 - February 2024

- Managed the Trust & Safety team with 5 direct reports in a full-stack engineering team.
- Researched, designed, and led development efforts to create a new system for scam detection for OpenDoor properties, which led to an almost 60% reduction in related fraud.
- Supported and expanded the internal machine learning model for identifying a risk score for potential and current homes using Snowflake AI and ML. Although the solution was not complete at the time of my departure, the components that were released allowed operations to evaluate the risk level of potential properties orders of magnitude faster, reducing the turnaround time from over a week to a couple of hours
- Directed development efforts for a new internal case management system that migrated workflows out from our legacy system, as well as enabled new workflows for our users. The new workflows allowed the business to shift from a 24+ hour delay in reviewing potential fraud cases to near-real time reviews. This rollout also included additional fraud detection rules that further reduced fraud by 43%.

SUMMARY OF EXPERIENCE

Veteran software architect and engineering leader with 17+ years of experience building, modernizing, and scaling complex enterprise systems. Currently serving as a Senior Engineering Manager at American Express, deliberately pivoting to focus exclusively on hands-on applied AI engineering. Combines the strategic, compliance-driven rigor of an enterprise leader with deep, active technical expertise in building agentic LLM workflows, retrieval-augmented generation (RAG), and multi-model orchestration.

TECHNICAL SKILLS

Highly experienced in C#, Go, Javascript (node.js, Typescript), Python

Highly skilled in relational databases (MSSQL, PostgreSQL, Snowflake) and non-relational databases

Experienced in common python data analysis & ML libraries such as pandas, numpy, scikit-learn, matplotlib, xgboost, seaborn, hyperopt, scipy

6+ Years of experience using both Azure and AWS using cloud native technologies

Carvana, Tempe, AZ — Engineering Manager

January 2020 - October 2022

- Managed 4 teams over the loan underwriting process with 4 direct reports and 18 indirect reports within Engineering (most companies would refer to this role as a *Senior Engineering Manager*, due to the multiple teams, and only having direct reports of other managers).
- Led a highly coordinated and collaborative effort to create a major rewrite of the entire underwriting system in order to enable greater scalability and automation. Overall system availability went from 91% to 98%. This increased availability led to an almost 18% increase of same day processing of purchases.
- Owned mission-critical Salesforce customer data integration that could not keep pace with Carvana's growth. Built monitoring, alerting, and disaster recovery from scratch, then rewrote the integration for 10x throughput—turning a brittle bottleneck into a scale-ready, zero-surprise backbone for customer service operations.
- Architected and led development teams for a centralized hiring orchestration hub, integrating PeopleSoft, Workday, and Greenhouse via webhooks, service bus, and REST APIs, replacing fragmented manual coordination across multiple teams. Designed event-driven synchronization and piloted LLM-based resume evaluation to accelerate candidate screening, establishing a scalable pipeline capable of reducing hiring workflow friction by an estimated 40%.
- Led the Underwriting efforts of an effort across a multi-department initiative to implement a centralized and automated communication system to automatically request and process additional documents needed to underwrite a sale. This effort led to an overall 19% reduction in the amount of times an underwriting case needed review, and a 13% reduction in total customer calls needed per sale.

Netflix, Los Angeles, CA — Senior Software Engineer

May 2019 - December 2019

- Individual contributor working on public facing press site (media.netflix.com), which is a multilingual site for global and regional releases to support a key focus within Netflix to create and promote new content for the streaming platform.
- Developed on an internal site to help publicists and content promoters quickly and consistently extend content on the media site. This system integrated with over 25 different internal and external systems to leverage a wealth of information already available within the Netflix ecosystem of services for upcoming titles.
- In addition to engineering responsibilities, regularly met with key stakeholders to gather insights on the current and future needs of the system.

Carvana, Tempe, AZ — Engineering Team Lead

March 2018 - May 2019

- Managed team of 5 engineering direct reports. over the title registration portion of the business (most companies would refer to this position as an *Engineering Manager*, due to the direct reports).
- Initiated effort on integrating with 3rd parties for submission and monitoring of electronic vehicle registrations with various state DMVs. At the time of the project 22 states were supported, and the number in registrations that were completed successfully and without subsequent intervention rose 36%.

Emailage, Tempe, AZ — Technical Lead, Senior Software Engineer

July 2017 - March 2018

- Technical lead on a team building out the next generation, high performance API for fraud detection. Performance targets were aggressively set, looking to replace the (then current) API in-processor time from 2 seconds to a P95 of 30 milliseconds, using the same core datasets for fraud evaluation.

APPLIED AI & ARCHITECTURAL PROJECTS

Doctoral Research Collaboration

- Research Collaboration (Political Science PhD Candidate): Designed and built an agentic LLM harness for automated analysis of political communiqués. Architected a multi-model workflow utilizing four frontier LLMs in parallel, each executing a self-critique cycle before submitting to a judge model for final scoring. Enabled systematic, replicable scoring of qualitative text at scale—replacing manual rubric application with autonomous, explainable AI agents. The rigor of the multi-agent workflow led to an average of over 90% alignment of Krippendorff's Alpha across the models.

Custom RAG Retrieval & Indexing Optimization System

- Architected and developed a modular Retrieval-Augmented Generation (RAG) framework designed to systematically benchmark and evaluate custom indexing architectures. Engineered the system to isolate the data ingestion pipeline, enabling rapid, empirical testing of varied dynamic text-chunking and vectorization strategies. This proof-of-concept specifically targets the optimization of semantic retrieval precision and context-window efficiency, ensuring that the underlying vector storage maximizes the recall, accuracy, and relevance of LLM-generated responses across complex unstructured data.

Custom Multi-Agent Workflow & Synthetic Data Harness

- Architected and engineered a proprietary, multi-agent orchestration harness from scratch to autonomously evaluate complex technical concepts and generate highly structured synthetic datasets. Designed the system to leverage interacting AI agents that reason over specialized contexts, execute iterative evaluation loops, and format high-quality outputs specifically tailored for downstream LLM fine-tuning. This framework demonstrates deep, hands-on capabilities in building resilient agentic systems, managing stateful multi-model workflows, and accelerating the deployment of specialized, fine-tuned models from concept to execution.

EDUCATION

Western Governors University, Salt Lake City, UT

Masters of Business Administration

Bachelors of Computer Science